

Blood Donation Platform

وثيقة مختصرة ومنسقة لمشروع تخرج يربط محتاجي الدم بالمتبرعين المناسبين في الوقت الفعلي، باستخدام ASP.NET Core و Angular مع Multi-Agent AI Pipeline للمطابقة الذكية والإشعارات الفورية.

نوع المشروع

Full Stack - مشروع تخرج

مدة التنفيذ

شهر واحد

الفريق

5 أعضاء

المعمارية

Clean Architecture
CQRS + Multi-Agent AI

Backend

ASP.NET Core 8 + SQL Server

Frontend

Angular 17

1. فكرة المشروع والجانب التجاري

2. المعمارية التقنية

3. الذكاء الاصطناعي - Multi-Agent AI Pipeline

4. خطة التنفيذ وتوزيع الفريق - شهر واحد

تم إعداد هذه النسخة لتحتوي فقط على الأقسام المطلوبة، مع الحفاظ على نفس الهوية البصرية: عناوين باللون الأحمر، جداول منظمة، ومسافات كافية لمنع تداخل النصوص.

1. فكرة المشروع والجانب التجاري

1.1 المشكلة التي يحلها المشروع

عند احتياج مريض إلى دم بشكل عاجل، غالباً يبدأ أهل المريض أو المستشفى في التواصل اليدوي مع الأقارب والمعارف للبحث عن متبرع مناسب. هذه الطريقة بطيئة، غير منظمة، ولا تضمن الوصول إلى متبرع قريب ومتوافق في الوقت المناسب. المشكلة الأساسية ليست فقط في نقص المتبرعين، بل في عدم وجود نظام سريع يربط بين الطلب والمتبرع المناسب بناءً على الفصيلة، الموقع الجغرافي، وتوفر المتبرع.

1.2 الحل المقترح

منصة رقمية تربط المرضى أو المستشفيات بالمتبرعين القريبين جغرافياً والمتوافقين طبياً. عند إنشاء طلب دم جديد، يقوم النظام بتشغيل مسار ذكي يحدد الأولوية، يفلتر المتبرعين، يرتبهم حسب المسافة، ثم يرسل لهم إشعارات فورية.

1.3 رحلة المستخدم الكاملة

#	الخطوة	التقنية المستخدمة
1	المريض أو المستشفى ينشئ طلب دم يحتوي على الفصيلة، الموقع، ودرجة الإلحاح.	Angular Form + REST API
2	النظام يرسل الطلب إلى مسار الذكاء الاصطناعي لتحديد الأولوية.	Triage Agent
3	فلتر المتبرعين غير المناسبين طبياً أو غير المتاحين.	Medical Validator Agent
4	ترتيب المتبرعين المناسبين حسب القرب من المستشفى.	Geo-Matching Agent + Haversine
5	إرسال إشعارات فورية للمتبرعين الأكثر مناسبة.	Notification Agent + SignalR
6	المتبرع يقبل الطلب، ويتم تحديث حالة الطلب فوراً.	Angular + PUT API
7	المريض أو المستشفى يتابع حالة الطلب من لوحة التحكم.	Dashboard + SignalR

1.4 أصحاب المصلحة

الدور	الوصف	ما الذي يفعله داخل المنصة؟
Patient	شخص يحتاج إلى دم.	ينشئ طلب دم ويتابع حالة الطلب.
Donor	متبرع مسجل بفصيلة دم وموقع معروف.	يرى الطلبات القريبة ويتلقى الإشعارات ويقبل الطلب المناسب.
Hospital	مستشفى أو مركز طبي.	يدير الطلبات ويؤكد اكتمال عملية التبرع.
Admin	مدير النظام.	يتابع المستخدمين، الطلبات، والإحصائيات العامة.

1.5 القيمة الاجتماعية والتجارية

المعيار	قبل المنصة	بعد المنصة
وقت الوصول إلى متبرع	قد يستغرق ساعات	دقائق في السيناريو المثالي
نطاق البحث	معارف شخصية فقط	متبرعون داخل نطاق جغرافي محدد
مطابقة الفصيلة	يدوية وغير مضمونة	تلقائية وتشمل الفصائل المتوافقة
متابعة الطلب	غير واضحة	تحديثات فورية ولوحة متابعة
سجل الطلبات	ورقي أو مشتمت	رقمي وقابل للبحث والتحليل

2. المعمارية التقنية

2.1 نظرة عامة على Clean Architecture

يعتمد المشروع على Clean Architecture لفصل المسؤوليات وتقليل الاعتماد بين الطبقات. القاعدة الأساسية أن الطبقات الداخلية تحتوي على منطق العمل ولا تعتمد على تفاصيل قواعد البيانات أو الواجهة أو الخدمات الخارجية.

الطبقة	المحتوى	المسؤولية
Domain Layer	Entities, Enums, Repository Interfaces	قواعد العمل الأساسية مثل المستخدم، طلب الدم، الفصيلة، وحالة الطلب.
Application Layer	Commands, Queries, DTOs, Interfaces, MediatR	تنظيم حالات الاستخدام وفصل عمليات القراءة عن الكتابة.
Infrastructure Layer	EF Core, SQL Server, Services, SignalR, AI Pipeline	تنفيذ الوصول للبيانات والخدمات الخارجية والإشعارات ومسار الذكاء الاصطناعي.
API Layer	Controllers, Middleware, Swagger, DI	استقبال الطلبات من الواجهة وتوجيهها إلى طبقة التطبيق.

2.2 نمط CQRS مع MediatR

CQRS يعني فصل أوامر الكتابة Commands عن استعلامات القراءة Queries. كل عملية لها طلب مستقل و Handler مسؤول عن تنفيذها، مما يجعل الكود أوضح وأسهل في الاختبار.

النوع	Class	Handler	الوظيفة
Command	RegisterCommand	RegisterHandler	إنشاء حساب مستخدم جديد.
Command	LoginCommand	LoginHandler	تسجيل الدخول وإرجاع JWT.
Command	CreateBloodRequestCommand	CreateBloodRequestHandler	إنشاء طلب دم وتشغيل مسار المطابقة الذكية.
Command	AcceptBloodRequestCommand	AcceptBloodRequestHandler	قبول الطلب بواسطة متبرع.
Query	GetNearbyRequestsQuery	GetNearbyRequestsHandler	عرض الطلبات القريبة من المتبرع.
Query	GetDashboardStatsQuery	GetDashboardStatsHandler	عرض إحصائيات لوحة التحكم.

3. الذكاء الاصطناعي - Multi-Agent AI Pipeline

الميزة الذكية في المشروع ليست مجرد AI Agent واحد، بل مسار من عدة Agents، كل Agent مسؤول عن خطوة محددة. هذا التصميم يجعل النظام أوضح في الشرح، أسهل في الاختبار، وأكثر قابلية للتطوير من وضع كل المنطق داخل كلاس واحد كبير.

3.1 الشكل العام لل Pipeline



3.2 مراحل عمل ال Agents

#	Agent	المسؤولية	التقنية/المنطق
1	Triage Agent	يحلل بيانات الطلب ويحدد الأولوية: عاجل، أو حرج.	Rules / OpenAI optional
2	Medical Validator Agent	يفلتر المتبرعين حسب توافق فصيلة، التوفر، وآخر تاريخ تبرع.	Blood Type Rules
3	Geo-Matching Agent	يحسب المسافة ويرتب المتبرعين من الأقرب إلى الأبعد.	Haversine Formula
4	Notification Agent	يبنى رسالة الإشعار ويرسلها للمتبرعين المناسبين فوراً.	SignalR + Notification Service

ملاحظة مهمة: لا يتم استخدام الذكاء الاصطناعي في كل خطوة بلا داعي. حساب المسافة وتوافق الفصائل يفضل أن يكونا قواعد واضحة وخوارزميات ثابتة، بينما يمكن استخدام OpenAI في تحليل النصوص أو توليد رسالة إشعار مناسبة عند الحاجة.

3.3 تفاصيل كل Agent

Triage Agent

يحدد أولوية الطلب بناءً على درجة الإلحاح، ملاحظات الحالة، وقت الطلب، وبيانات المستشفى. الناتج يكون مستوى أولوية واضح يساعد النظام على التعامل مع الحالات الحرجة بسرعة.

Medical Validator Agent

يستبعد المتبرعين غير المناسبين طبياً مثل غير المتاحين، غير المتوافقين في فصيلة، أو من لديهم تاريخ تبرع حديث لا يسمح بالتبرع مرة أخرى.

Geo-Matching Agent

يستخدم إحداثيات المستشفى والمتبرعين لحساب المسافة باستخدام معادلة Haversine، ثم يرتب النتائج حسب الأقرب.

Notification Agent

يرسل إشعارات فورية للمتبرعين المناسبين باستخدام SignalR، ويعرض الفصيلة، درجة الإلحاح، اسم المستشفى، والمسافة التقريبية.

3.4 جدول توافق فصائل الدم

الفصيلة المطلوبة	الفصائل التي يمكنها التبرع
O-	O- فقط
O+	O+, O-
A-	A-, O-
A+	A+, A-, O+, O-
B-	B-, O-
B+	B+, B-, O+, O-
AB-	AB-, A-, B-, O-
AB+	كل الفصائل

هذا الجزء يمكن شرحه في العرض على أنه Multi-Agent Decision Pipeline وليس مجرد استخدام مباشر لـ GPT. هذه صياغة أقوى أكاديمياً لأنها توضح أين نستخدم القواعد الطبية، أين نستخدم الخوارزميات، وأين يمكن استخدام الذكاء الاصطناعي.

4. خطة التنفيذ وتوزيع الفريق - شهر واحد

4.1 خطة شهر واحد

الأسبوع	الهدف	المهام الأساسية	المخرجات
الأسبوع الأول	التأسيس والمعمارية	إعداد المشروع، إنشاء Clean Architecture، بناء Domain Entities، إعداد قاعدة البيانات، وتطبيق التسجيل والدخول.	مشروع يعمل Auth + أساسي
الأسبوع الثاني	طلبات الدم والمطابقة الأساسية	بناء Commands/Queries، إنشاء طلب دم، عرض الطلبات القريبة، حساب المسافة، وتطبيق صلاحيات الأدوار.	تدفق طلب دم كامل مبدئي
الأسبوع الثالث	Multi-Agent AI Pipeline والإشعارات	بناء Triage، Medical Validator، Geo-Matching، و Notification Agent، مع تفعيل SignalR.	مطابقة ذكية + إشعارات فورية
الأسبوع الرابع	الواجهة، الاختبار، والتسليم	تحسين واجهات Angular، بناء لوحة التحكم، اختبار السيناريوهات الأساسية، تجهيز العرض النهائي والوثائق.	نسخة MVP قابلة للعرض

4.2 توزيع الأدوار على فريق مكون من 5 أعضاء

العضو	الدور الأساسي	المسؤوليات
1	Backend Lead	بناء Domain و Application Layer وتنفيذ CQRS.
2	Database / Infrastructure	إعداد EF Core، المستودعات، قاعدة البيانات، والخدمات الأساسية.
3	Frontend Lead	بناء صفحات Angular، تسجيل الدخول، طلب الدم، الطلبات القريبة، ولوحة التحكم.
4	AI & Real-Time Integration	تنفيذ Multi-Agent Pipeline، الإشعارات، SignalR، والربط مع الواجهة.
5	Testing / Documentation / Deployment	اختبار السيناريوهات، تجهيز README، التوثيق، وتجهيز عرض المشروع.

لأن مدة التنفيذ شهر واحد فقط، الأفضل اعتبار هذه النسخة MVP: التركيز على إنشاء الطلب، المطابقة الذكية، الإشعارات الفورية، ولوحة متابعة بسيطة. أي خصائص إضافية يمكن وضعها كتحسينات مستقبلية.